

sdgggroup_challenge

Explicación técnica detallada del motor entregado — con foco en las tres formas de ejecutar `main.py` y su resultado real — y propuesta de arquitectura extendida sobre Google Cloud con Airflow y CI/CD.

1. Introducción y alcance

Este documento es la versión detallada del punto 6 del enunciado. La presentación (deck 1) resume los puntos más importantes; este documento explica cada uno con el resultado real obtenido al ejecutarlo, y desarrolla en profundidad la arquitectura extendida que pide el enunciado.

El enunciado pedía desarrollar un motor de procesamiento dirigido por metadatos, demostrar su comportamiento con dos cargas de datos distintas (2024 y 2025), construir dos comprobaciones adicionales sobre los datos crudos, y documentar cómo se industrializaría una solución de este tipo en una arquitectura cloud con Airflow y CI/CD.

#	Qué pide el enunciado	Estado	Dónde
1	CLI: parseo y validación de argumentos de lanzamiento	Hecho	engine/cli.py
2	Parseo y validación de metadata.json; construcción del DAG en memoria	Hecho	engine/metadata_schema.py, engine/graph.py
3	Motor de ejecución genérico sobre Spark + Delta Lake	Hecho	engine/executor.py y módulos de dispatch
4	Ejecutar con 2024 y después con 2025, demostrando overwrite / append / merge	Hecho	Sección 3.1 de este documento
5	Comprobaciones (a) diff de filas y (b) calidad de dato, sobre los CSV crudos	Hecho	checks/, sección 4
6	Documento de arquitectura extendida: cloud + Airflow + servicios + CI/CD (sin IaC)	Este documento	Sección 6

Dos añadidos no pedidos explícitamente pero que facilitan la revisión de todo lo anterior: un flag `--dry-run` que imprime el plan de ejecución (nodos, transformaciones, instrucción Spark de cada una, orden topológico) sin tocar nada, y un subcomando `main.py check ...` que expone las dos comprobaciones del punto 5 desde el mismo punto de entrada. Ambos se detallan en la sección 3.

2. Arquitectura del motor entregado

Un único motor genérico, dirigido enteramente por `metadata.json`: nada del comportamiento de un dataflow concreto está hardcodeado en el código.



2.1 Módulos y responsabilidad de cada uno

Módulo	Responsabilidad
<code>engine/cli.py</code>	Argumentos del comando: <code>--metadata</code> , <code>--year</code> , <code>--tables-base-path</code> , <code>--dry-run</code> . Cada fallo de validación propio devuelve un <code>exit_code</code> distinto.
<code>engine/metadata_schema.py</code>	Modelos Pydantic discriminados por <code>type</code> para <code>inputs/transformations/outputs</code> ; validación estructural completa del fichero.
<code>engine/graph.py</code>	<code>build_graph(s)</code> : construye el DAG en memoria, separando aristas de datos (<code>input</code>) de aristas de espera (<code>waits</code>).
<code>engine/topology.py</code>	<code>topological_sort</code> : DFS de tres colores sobre el grafo combinado; <code>GraphCycleError</code> nombra la secuencia exacta de nodos si hay ciclo.
<code>engine/templating.py</code>	Resolución genérica de <code>{{ variable }}</code> en cualquier valor de <code>config</code> (hoy: <code>{{ year }}</code>).
<code>engine/paths.py</code>	Rebase de los paths pseudo-absolutos de <code>metadata.json</code> (<code>/data/...</code>) bajo la raíz del proyecto, portable entre Windows y POSIX.
<code>engine/spark_session.py</code>	<code>create_spark_session(with_delta=...)</code> : una única <code>SparkSession</code> ; <code>with_delta=False</code> evita la resolución Ivy/Delta cuando no hace falta (usado por <code>checks</code>).
<code>engine/readers.py</code>	Lectura genérica por <code>format</code> (<code>spark.read.format(...).options(...).load(...)</code>), con <code>templating</code> ya resuelto.

engine/transformations.py	Dispatch filter / add_fields / group, cada uno una operación Spark encadenada.
engine/writers.py	Dispatch file+save_mode / table+save_mode (Delta: append directo, merge por primary_key con bootstrap a overwrite).
engine/executor.py	Orquestador: detecta hojas, las dispara en orden topológico, resuelve cada rama recursivamente con memoización y .cache() en los puntos de fan-out.
engine/describe.py	Soporte de --dry-run: describe el grafo y la instrucción Spark de cada nodo sin importar pyspark ni crear sesión alguna.

2.2 Decisiones de diseño que hacen el motor genérico

DAG DIRIGIDO POR METADATA

Aristas de datos y de espera en listas separadas; un único orden topológico sobre el grafo combinado, con detección de ciclos explícita.

MEMOIZACIÓN + CACHE SELECTIVO

Cada nodo se resuelve una sola vez; se cachea automáticamente solo cuando alimenta a más de un consumidor directo (fan-out real).

`WAITS` FUERZA UNA ACCIÓN REAL

Un input que declara waits fuerza la escritura real (no solo el DAG lazy) del nodo esperado antes de leer de esa misma ruta.

DELTA POR PATH, SIN METASTORE

append escribe directo; merge hace upsert por primary_key, con bootstrap a overwrite en la primera ejecución sobre un path sin tabla Delta todavía.

3. Las tres formas de ejecutar main.py

Foco de este documento: qué hace cada forma de invocar el programa, qué pasa por debajo, y el resultado real obtenido — no simulado — al ejecutarlo contra los datos del enunciado.

3.1 Ejecución estándar del pipeline

`python main.py --metadata metadata.json --year <YYYY>` — parsea y valida `metadata.json`, construye el DAG, crea la `SparkSession` y ejecuta de verdad: lee, transforma y escribe los 4 outputs del dataflow prueba-acceso.

Se ejecutó primero con `data2024.csv` y, después, con `data2025.csv` contra el mismo `metadata.json`, para demostrar los tres `save_mode`. La evidencia se capturó con `scripts/verify_outputs.py`, que lee cada destino tal como quedó en disco/Delta y, para las tablas, imprime `DeltaTable.history()`.

Output	save_mode	Filas tras 2024	Filas tras 2025	Lectura
write_last_file	overwrite	15.410	15.410	Reemplaza, no acumula
write_historic_file	append (partición load_date)	15.410	30.820	Acumula entre ejecuciones
write_delta_raw	table, append	15.410 (v0, 2026-09-26)	30.820 (v1, +15.410, 2026-10-01)	Acumula, no deduplica
write_delta_merge	table, merge	15.410 (v0, bootstrap overwrite)	15.410 (v1: 0 insertadas, 15.410 actualizadas)	Upsert por clave, sin duplicar

Detalle de `operationMetrics` de la versión 1 (tras ejecutar con 2025) de `write_delta_merge`, capturado por `verify_outputs.py` vía `DeltaTable.history()`:

```
v1 MERGE (2026-10-01 22:40:45.773)
numSourceRows      = 15410
numTargetRowsMatchedUpdated = 15410
numTargetRowsInserted  = 0
numTargetRowsDeleted   = 0
numTargetFilesRemoved  = 200 # los 200 ficheros del bootstrap v0
numTargetFilesAdded    = 1
executionTimeMs       = 10139
```

Todas las claves (provincia, municipio, sexo) de la agregación ya existían en v0, por eso el merge actualiza las 15.410 filas y no inserta ninguna -- es exactamente el comportamiento de un upsert sobre datos que se re-cargan cada año.

La columna `load_date` registra la fecha real de la ejecución (`current_date()`), no el año de los datos: por eso las particiones del `historic` son `2026-09-26` y `2026-10-01`, no `2024/2025`. Es una limitación conocida -- dos ejecuciones el mismo día caerían en la misma partición -- discutida en la sección 6.3.

3.2 --dry-run

`python main.py --metadata metadata.json --year <YYYY> --dry-run` — valida y mapea el grafo exactamente igual que una ejecución real (misma validación, mismo `topological_sort`), pero se detiene antes de llamar a `create_spark_session()`: nunca se crea una `SparkSession` ni se ejecuta ninguna lectura, transformación o escritura. El módulo que lo implementa, `engine/describe.py`, está deliberadamente libre de cualquier import de `pyspark` (ni siquiera transitivo), precisamente para que imprimir un plan no pague el arranque de Spark.

Para cada nodo imprime, además de su configuración resuelta, la instrucción Spark literal que un run real construiría y ejecutaría. Salida real (resumida), ejecutada con `--year 2025`:

```
$ python main.py --metadata metadata.json --year 2025 --dry-run
Parsed metadata file: metadata.json
year=2025 tables_base_path=/data/demo/output/tables
Dataflow 'prueba-acceso': 9 nodes (7 data edges, 1 wait edges)
```

--dry-run: no SparkSession created, nothing executed.

Inputs:

- demo_data [input/file]
spark = spark.read.format("csv").options(header="true", delimiter=";").load("../data/demo/input/poblacion2025.csv")
- parquet_data [input/file]
waits for = write_last_file (must finish writing before this is read)
spark = spark.read.format("parquet").load("../data/demo/output/last")

Transformations:

- new_fields [add_fields] spark = demo_data.withColumn("domain", F.expr("demography"))
.withColumn("load_date", F.expr("current_date()"))
- filter_rows [filter] spark = new_fields.filter("sexo != 'Ambos sexos' and municipio != 'N/A'")
- group_by_fields [group] spark = parquet_data.groupBy("provincia", "municipio", "sexo")
.agg(F.expr("sum(total) as total_ambos_sexos"))

Outputs (final actions):

- write_historic_file spark = filter_rows.write.format("parquet").mode("append")
.partitionBy("load_date").save("../output/historic")
- write_last_file spark = filter_rows.write.format("parquet").mode("overwrite")
.save("../output/last")
- write_delta_raw spark = group_by_fields.write.format("delta").mode("append")
.save("../tables/raw_demo")
- write_delta_merge spark = if DeltaTable.isDeltaTable(spark, "../tables/demo"):
DeltaTable.forPath(...).alias("target")
.merge(group_by_fields.alias("source"), "target.provincia`=`...")
.whenMatchedUpdateAll().whenNotMatchedInsertAll().execute()
else:
group_by_fields.write.format("delta").mode("overwrite").save(...)

Execution order (topological): demo_data -> new_fields -> filter_rows ->
write_historic_file -> write_last_file -> parquet_data -> group_by_fields ->
write_delta_raw -> write_delta_merge

La captura completa, sin resumir, está en docs/evidence/ y se reprodujo también tras cada corrección de código a lo largo del proyecto.

Para el save_mode=merge, decidir entre la rama de merge y la de bootstrap overwrite depende de si ya existe tabla Delta en ese path -- una comprobación real contra el filesystem/Delta log que --dry-run no puede ni debe hacer sin crear sesión. Por eso se muestran ambas ramas en vez de adivinar cuál correría.

3.3 check diff-rows / check data-quality

Expone las dos comprobaciones del punto 5 (detalladas en la sección 4) desde el mismo punto de entrada. El dispatch intercepta check *antes* de parse_cli_args, así que no afecta en nada a la validación ni a la ejecución normal del pipeline -- verificado explícitamente con tests y con check en posiciones no válidas de la línea de comandos.

```
$ python main.py check diff-rows
Setting default log level to "WARN".
Rows in 2024: 23268
Rows in 2025: 23268
Unchanged rows: 23260
Changed rows (same municipio+sexo, different total): 8
New rows in 2025: 0
Rows removed vs. 2024: 0
Total population 2024: 10914656
Total population 2025: 10914652
Net change: -4

Full detail (8 rows) written to: ../checks/output/diff_2024_2025.csv
```

```
$ python main.py check data-quality
Setting default log level to "WARN".
Municipios/key rows checked (provincia+municipio): 7756
Violations of 'Ambos sexos' != 'Hombres' + 'Mujeres': 2
Sexo categories missing for some municipio: 0
Total issues detected: 2

Full detail (2 rows with issues) written to: ../checks/output/data_quality_issues.csv
```

El ruido ERROR ShutdownHookManager / CORRECTO: el proceso... ha sido terminado. que aparece tras cada ejecución en Windows es benigno: es el propio sistema operativo limpiando el directorio temporal de la JVM al cerrar Spark, no un fallo del programa -- se omite aquí por claridad.

4. Comprobaciones del punto 5, en detalle

Ambas comprobaciones operan sobre los CSV crudos de `assets/` -- nunca sobre los outputs del pipeline -- por lo que su resultado es idéntico sin importar cuántas veces, en qué orden, o con qué `-year` se haya ejecutado `main.py`. Reutilizan `engine.spark_session.create_spark_session(..., with_delta=False)` para no pagar la resolución de Ivy/Delta cuando no hace falta.

4.1 (a) Diff de filas 2024 → 2025 — `checks/diff_rows.py`

Clave: (`provincia`, `municipio`, `sexo`) -- la clave natural del fichero, aparece exactamente una vez por año. `full_outer join` entre los dos años sobre esa clave; cada fila se clasifica como `changed` (mismo clave, `total` distinto), `added` (clave solo en 2025) o `removed` (clave solo en 2024); las filas `unchanged` se descartan. El CSV de detalle incluye la clave, el tipo de cambio y el valor antiguo y nuevo de `total`.

4.2 (b) Calidad de dato — `checks/data_quality.py`

Granularidad: (`provincia`, `municipio`) -- la misma clave de (a) menos `sexo`, porque esta comprobación compara precisamente los tres valores de `sexo` entre sí dentro de esa clave. Para cada municipio verifica que `'Ambos sexos' == 'Hombres' + 'Mujeres'`; marca `sum_mismatch` si no cuadra, o `missing_category` si falta alguna de las tres categorías. Es genérica sobre el fichero de entrada (`--input` acepta cualquier `poblacionXXXX.csv`) y termina con exit code 20 si encuentra incumplimientos, para poder usarse como gate en CI.

Ambas quedaron conectadas al CLI (`main.py check diff-rows / check data-quality`) y cubiertas por tests dedicados que reutilizan la fixture `spark` de `conftest.py`.

5. Tests y calidad

64 tests, pytest, 100% en verde.

MOTOR

`engine.cli` (incl. `--dry-run`),
`metadata_schema`, `graph`, `topology`, `paths`,
`describe` -- este último con un end-to-end que
verifica que `--dry-run` nunca crea una
`SparkSession` ni cambia `mtime`/tamaño de
ningún fichero de salida existente.

CHECKS + CLI

`diff_rows`, `data_quality`, y el dispatch de
`main.py check` -- incluida la comprobación
explícita de que `check` en cualquier posición que
no sea la primera deja el comando sin reconocer
(`argparse`, `exit 2`), nunca ejecutando el pipeline
por accidente.

Un detalle real de esta prueba: en Python 3.14, el `cloudpickle` que trae `pyspark==3.5.1` no sabe serializar el cierre que necesita `spark.createDataFrame(lista, schema=...)` -- entra en recursión infinita en vez de lanzar un error legible. Los fixtures de `checks/` se reescribieron para escribir un CSV real en `tmp_path` y leerlo con la función de carga ya probada (ruta 100% JVM), en vez de depender de esa llamada.

6. Arquitectura extendida (GCP + Airflow + CI/CD)

El enunciado pide presentar cómo se implantaría un motor como este -- asumiendo lecturas, transformaciones y escrituras más amplias que las de esta prueba -- en una arquitectura de datos sobre una cloud habitual, con Airflow como orquestador y el resto de servicios necesarios para industrializarlo, incluyendo CI/CD. Sin IaC, tal como pide explícitamente el enunciado.

6.1 Supuestos: funcionalidades extendidas

El motor de la prueba ya es genérico por diseño (dispatch por type/format), así que extenderlo a producción es añadir readers/writers nuevos, no rediseñar el núcleo:

- **Lectura:** JDBC (bases de datos relacionales), Avro/Parquet en GCS, APIs REST paginadas, colas (Pub/Sub), ficheros en SFTP.
- **Transformación:** las mismas tres (filter/add_fields/group) más joins entre dataflows, window functions, y validaciones de calidad como transformación de primera clase (hoy viven en checks/, aparte).
- **Escritura:** además de Delta por path, tablas registradas (BigQuery, metastore de Dataproc), colas de salida, APIs de terceros.

6.2 Diagrama de arquitectura (GCP)



El motor se empaqueta como una imagen de contenedor y se ejecuta sobre **Dataproc Serverless para Spark** (sin clúster persistente que gestionar) o, si se necesita autoscaling más fino o Spark Structured Streaming, sobre un clúster Dataproc en GKE. **GCS** sustituye al sistema de ficheros local de la prueba: una zona raw/landing (copia fiel de la fuente), una zona de trabajo, y una zona curated en formato Delta (o tablas nativas de **BigQuery** para el consumo analítico). metadata.json pasa a vivir

versionado en el propio repositorio Git, y **Data Catalog** lo indexa para que el gobierno del dato (qué dataflow produce qué tabla, con qué clave) sea consultable fuera del código.

6.3 Orquestación con Airflow

QUÉ GENERA EL DAG

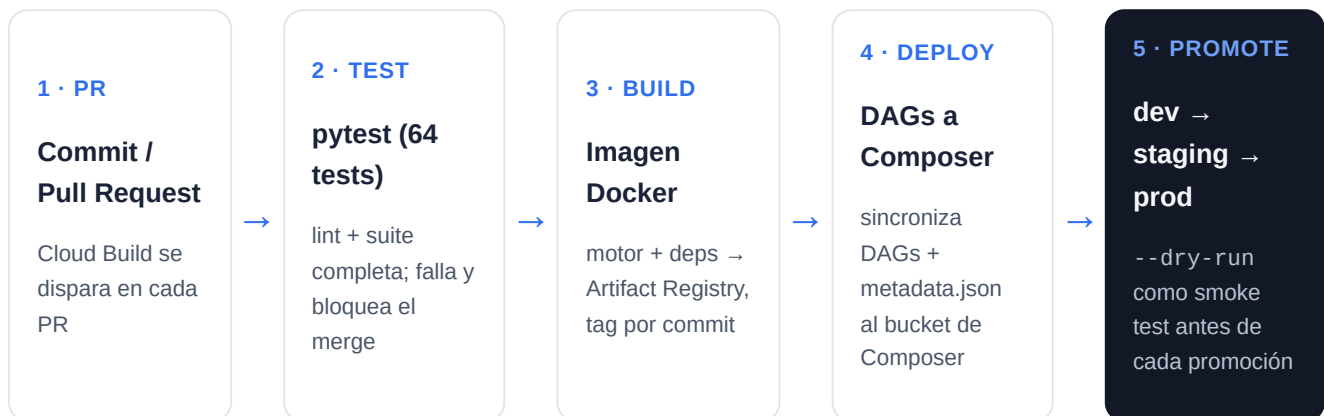
Composer lee cada dataflow de `metadata.json` y genera un `DataprocsSubmitJobOperator` (o `KubernetesPodOperator`) que lanza el motor, con `--metadata/--year` resueltos por plantilla de Airflow (`{{ ds }}`, etc.).

QUÉ SIGUE IGUAL

El DAG interno de dependencias (`input/waits`) lo sigue resolviendo el propio motor dentro de una tarea -- Airflow no necesita un task por nodo del grafo, solo uno por dataflow (o por grupo de salidas independientes que convenga paralelizar).

Airflow añade lo que el motor de la prueba no necesitaba resolver por sí mismo: **scheduling** (periodicidad por dataflow), **reintentos y SLAs**, **backfill** parametrizado por `--year/fecha`, **sensores** de llegada de fichero o de partición en BigQuery antes de lanzar una tarea, **alertas** (Slack/email en fallo), y un paso de `--dry-run` como validación previa automática antes de cada ejecución real -- exactamente la garantía que ya ofrece hoy sin crear `SparkSession` ni tocar datos. También resuelve la limitación de `load_date` señalada en 3.1: con Airflow, la partición pasa a ser la `execution_date` lógica del DAG run, no `current_date()`, así que dos reintentos el mismo día ya no colisionan.

6.4 CI/CD



Tres entornos (dev/staging/prod), cada uno con su propio bucket de Composer, su propio `--tables-base-path` y sus propias credenciales en Secret Manager. El paso 2 es el mismo `pytest` que ya corre hoy en local -- nada que inventar para CI. El paso 5 usa `--dry-run` contra el `metadata.json` del entorno destino como última comprobación automática antes de promover: si el grafo no valida o hay un ciclo, la promoción se detiene ahí, sin haber tocado un solo dato.

Por instrucción explícita del enunciado, este documento no desarrolla la creación ni el despliegue de estos servicios cloud mediante Infraestructura como Código (Terraform, Deployment Manager, etc.).

7. Conclusión

Los cinco primeros puntos del enunciado están hechos, probados con 64 tests, y verificados con datos reales -- `overwrite`, `append` y `merge` demostrados con números concretos de `DeltaTable.history()`, no solo afirmados. El sexto punto es la propuesta de esta sección: el mismo motor, sin rediseño de su núcleo, encaja en una arquitectura GCP con Dataproc Serverless, Cloud Composer como orquestador y un pipeline de CI/CD de cinco pasos que usa `--dry-run` como gate de seguridad antes de cada promoción.